

Table 1: The differences between classical and quantum computing

Parameter	Classical approach	Quantum approach
Logic	Binary logic is the foundation of classical programming, which manipulates bits in a deterministic manner while operating in the domain of zeros and ones	The fundamental ideas of quantum mechanics, a branch of physics that explains how atoms and subatomic particles behave, give rise to quantum programming
Unit	Bit	Qubit (quantum bit)
Representation of data	Bits are in one of two states: 0 or 1	Qubits can simultaneously represent 0 and 1 when they are in a superposition
Processing	Processing is in a sequential manner, handling each bit separately	Processing is in parallel because of entanglement and superposition
Computation power	Less in comparison of quantum computing. Effective for many different tasks, but difficult to attain exponential speedup	Possibility of exponential speedup in certain problem categories like factorisation and search
Manipulation of data	Classical gates like AND, OR, NOT are used	Uses quantum gates (e.g., Hadamard, CNOT, Pauli gates) for manipulation
Designing of an algorithm	Algorithms have a defined logical flow and are deterministic	Probabilities must be taken into account by algorithms, which are frequently non-deterministic
Circuit structure	Classical structure, which is stable, deterministic and does not require probabilistic interpretation	Quantum circuit; complex and probabilistic, requiring multiple runs for meaningful results
Error correction method	Advanced quantum error correction	Traditional error correction
Scalability	Highly scalable	Scalability is challenging because of error rates, decoherence, and sensitivity to environmental factors
Hardware requirements	Operates on silicon-based processors, which include powerful CPUs, GPUs, and microcontrollers	Requires specific hardware, like photonic systems, ion traps, or superconducting circuits
Types of problems addressed	Good for deterministic algorithms, database operations, and arithmetic tasks	Ideal for problems involving quantum simulations, cryptography, and large scale optimisation
Programming languages	Makes use of popular languages like Python, C++, Java, and numerous other high level programming languages	Quantum-specific languages with a focus on quantum gates including Qiskit, Cirq, and Q#
Examples	Personal computers, supercomputers, mainframes, cloud-based servers	IBM's quantum computers, Google's Sycamore, D-Wave's quantum annealers
Use cases	Web apps, machine learning, engineering simulations, general computing, and daily tasks-related applications	Materials science, quantum simulations, cryptography, specific AI algorithms, and optimisation issues

speedup, and quantum hardware are vital to understand quantum computing effectively. As we have seen, quantum computing is different from classical computing in so many ways including processing, computational power, data manipulation, design of an algorithm, circuit structure, error correction method, scalability, and hardware requirements. **END** 

References

- <https://www.techtarget.com/searchdatacenter/tip/Classical-vs-quantum-computing-What-are-the-differences>
- <https://www.ibm.com/topics/quantum-computing>

By: Dr Dharmendra Patel

The author is a professor at Smt Chandaben Mohanbhai Patel Institute of Computer Applications, Gujarat. His areas of interest are data mining, web mining, image processing, Internet of Things and software engineering.